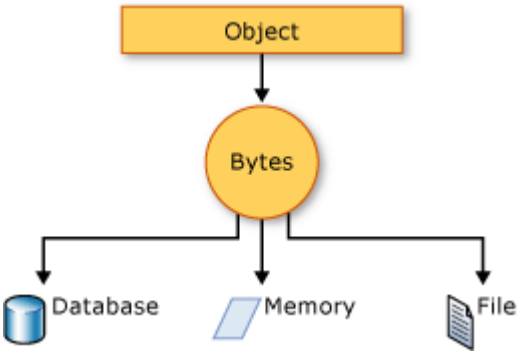
Veri Gönderimi & Serialization

Bu bölümde, oyun verilerinin ağ üzerinden nasıl taşındığını ve farklı formatların avantajlarını inceleyeceğiz. Serialization (serileştirme), karmaşık veri yapılarını (nesneler, listeler, sözlükler) ağ üzerinden gönderilebilecek bayt (byte) dizisine dönüştürme işlemidir. Bu dönüşümün nasıl yapıldığı, oyunun performansını, bant genişliği kullanımını ve geliştirme kolaylığını doğrudan etkiler.



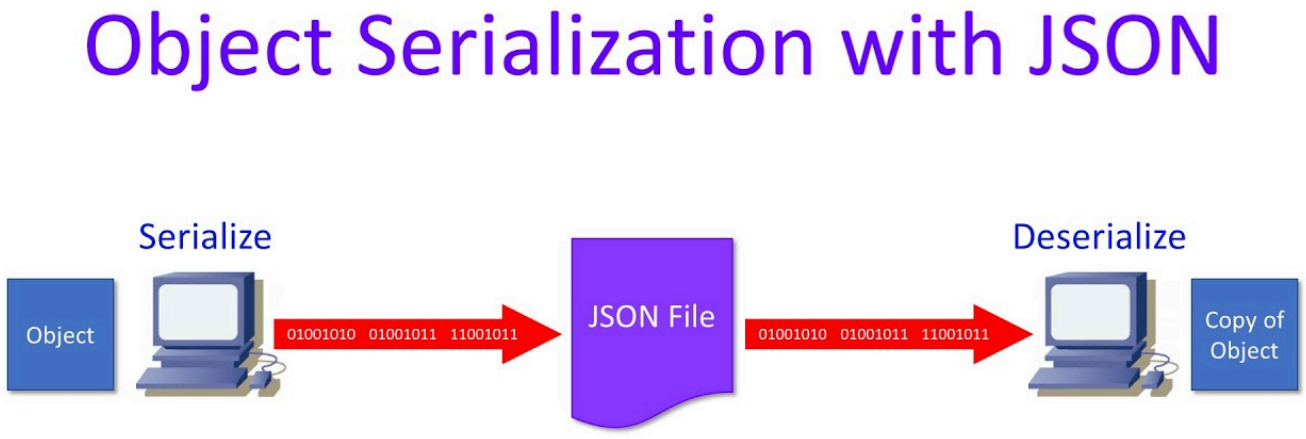
---

1. JSON vs Binary Data (İkili Veri)

Oyun verilerini serileştirmek için iki ana yaklaşım vardır: İnsan tarafından okunabilir metin tabanlı formatlar (JSON, XML) ve makine tarafından optimize edilmiş ikili (binary) formatlar.

1.1. JSON (JavaScript Object Notation)

- **Tanım:** Verileri anahtar-değer (key-value) çiftleri halinde düzenleyen, metin tabanlı bir veri değişim formatıdır.



- **Örnek JSON Verisi:**

```
{
  "playerId": 101,
  "position": {
    "x": 125.5,
    "y": 75.2,
    "z": 0.0
  },
  "rotation": 90.0,
  "health": 80,
  "isAlive": true
}
```

Avantajları	Dezavantajları
İnsan tarafından okunabilir ve anlaşılabilir.	Metin tabanlı olduğu için **fazla bant genişliği** tüketir.
Hata ayıklama (debugging) çok kolaydır.	Parse etmek (ayrıştırmak) işlemci gücü gerektirir.
Çoğu programlama dilinde yerleşik kütüphane desteği vardır.	Veri tipleri sınırlıdır (tarih, binary veri gibi tipleri doğrudan desteklemez).
Protokol geliştirme ve değiştirme esnektir.	Alan adları (key) her pakette tekrar gönderilir, bu da ekstra yük oluşturur.

1.2. Binary Data (İkili Veri)

- **Tanım:** Verilerin, bellek üzerindeki ham (raw) hallerine en yakın şekilde, sıkıştırılmış bayt dizileri olarak gönderilmesidir. Özel formatlar (Protobuf, MessagePack) veya elle yapılan bayt sıralaması (manual byte packing) ile uygulanır.
- **Örnek Binary Temsili (Kavramsal):**

```
[ID][X][Y][Z][ROTATION][HEALTH][FLAGS]
4B  4B 4B 4B  4B        2B       1B
```

Burada her alan kaç byte yer kaplayacağı önceden belirlenmiştir.

Avantajları	Dezavantajları
Çok **daha az bant genişliği** kullanır (JSON'a göre %50-80 daha küçük).	İnsan tarafından okunamaz, hata ayıklama zordur.
Parse etme (okuma/yazma) işlemi **çok hızlıdır** (doğrudan bellek kopyalama).	Alan sıralamaları ve veri tipleri önceden sabitlenmeli; esneklik azdır.
Veri tipleri tam kontrol altındadır (float, int, bool vs. net ayrışır).	Versiyonlama (field ekleme/çıkarma) zordur. Uyumsuzluklara neden olabilir.
Ağ trafiğini optimize etmek için idealdir.	Geliştirme süresi uzar, ek kodlama gerektirir.

1.3. Oyunlarda Tercih ve Hibrit Yaklaşım

Veri Türü	Tercih	Örnek
Anlık oyun verileri (Konum, Atış, Hasar)	**Binary**	Oyunun her frame'inde gönderilen veriler. Hız ve boyut kritik.
Oyun içi metin mesajları (Chat)	**JSON veya Metin**	Sohbet mesajları; okunabilirlik önemli, hız kritik değil.
Yapılandırma dosyaları	**JSON**	Başlangıç ayarları, oyun kuralları.
Matchmaking / Lobi bilgileri	**JSON**	HTTP API üzerinden gönderilir.
Hibrit (En yaygın oyun yaklaşımı)	**Binary + JSON**	Oyun verileri binary ile hızlı, diğer tüm metin bazlı işlemler JSON ile iletilir.

2. Mesaj Formatı Tasarımı

Herhangi bir ağ uygulamasında, gönderilen verinin ne olduğunu anlamak için bir mesaj formatı (protokol) belirlemek zorunludur. Bir mesaj formatı şunları içermelidir:

2.1. Temel Mesaj Bileşenleri

1. **Header (Başlık):** Mesajın türünü, uzunluğunu ve diğer meta verileri içeren sabit kısım.
   - `messageType` (1-2 byte): Hangi işlemin yapılacağını belirtir. (Örn: `MOVE`, `SHOOT`, `STATE_UPDATE`)
   - `messageLength` (2-4 byte): Mesajın toplam uzunluğu (header dahil veya body hariç).
   - `sequenceNumber` (2-4 byte): Paketlerin sırasını takip etmek için.
   - `timestamp` (4-8 byte): Gönderim zamanı.
2. **Body (Gövde):** Asıl verinin bulunduğu kısım. Serileştirilmiş JSON veya binary veridir.

2.2. Örnek Mesaj Yapısı (Tasarım)

C2S (Client-to-Server) - Move Message:

```
| 0x01 (TYPE) | 0x0016 (LENGTH) | 0x0001 (SEQ) | [Position JSON veya
Binary] |
```

- `0x01`: "MOVE" mesajı olduğunu belirtir.
- `0x0016`: Body uzunluğu (22 byte).
- `0x0001`: Sıra numarası.

S2C (Server-to-Client) - State Update Message:

```
| 0x02 (TYPE) | 0x0044 (LENGTH) | 0x1234 (SEQ) | [Tüm Oyuncuların
Durumları] |
```

- `0x02`: "STATE_UPDATE" mesajı.
- `0x0044`: 68 byte uzunluğunda bir body var.

2.3. Prefix (Önek) ile Mesaj Sınırlandırma (Framing)

TCP, akan veri (stream) olduğu için mesajların nerede başlayıp nerede bittiğini bilmeliyiz. Bunun için mesaj öncesinde uzunluk bilgisi göndermek standarttır.

```
[4 Byte: Mesaj Uzunluğu] [Mesaj Body (JSON veya Binary)]
```

Alıcı taraf önce 4 byte okur, mesaj boyutunu öğrenir, o kadar byte okur ve sonra bir sonraki mesaja geçer. Bu, TCP'de "message framing" (mesaj çerçeveleme) olarak adlandırılır.

---

3. Uygulama: Player Position Mesajı Gönderme (JSON ile)

Bu uygulamada, bir oyuncunun konum bilgisini (X, Y, Z koordinatları, rotasyon, oyuncu ID) JSON formatında serileştirerek bir UDP sunucusuna göndereceğiz.

Senaryo: İstemci, sürekli değişen oyuncu konumlarını (örneğin saniyede 10 kez) JSON paketleri halinde sunucuya gönderir. Sunucu bu paketleri alır, içindeki verileri ayrıştırır (parse) ve ekrana yazdırır.

3.1. Python ile JSON Serileştirme ve UDP Gönderimi (client_position.py)

```python
import socket
import json
import time
import random

# Sunucu bilgileri
HOST = '127.0.0.1'
PORT = 12345
BUFFER_SIZE = 1024

def create_position_message(player_id, x, y, z, rotation):
    """Oyuncu konum verisini JSON formatında oluşturur."""
    message = {
        "type": "POSITION_UPDATE",
        "timestamp": int(time.time() * 1000),  # Unix timestamp in ms
        "data": {
            "playerId": player_id,
            "position": {
                "x": round(x, 2),
                "y": round(y, 2),
                "z": round(z, 2)
            },
            "rotation": round(rotation, 2)
        }
    }
    return message
```

```
def simulate_movement():
    """Oyuncu hareketini simüle eder ve JSON mesajları gönderir."""
    # UDP soketi oluştur
    client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

    player_id = 101
    x, y, z = 0.0, 0.0, 0.0
    rotation = 0.0

    print(f"Oyuncu {player_id} konum gönderimi başladı. (Sunucu: {HOST}:
{PORT})")

    try:
        for i in range(20):  # 20 mesaj gönder
            # Rastgele hareket simülasyonu
            x += random.uniform(-2.0, 2.0)
            y += random.uniform(-1.0, 1.0)
            z += random.uniform(-0.5, 0.5)
            rotation = (rotation + random.uniform(-10.0, 10.0)) % 360

            # JSON mesajı oluştur
            msg_dict = create_position_message(player_id, x, y, z,
rotation)
            json_message = json.dumps(msg_dict)  # JSON -> String
            json_bytes = json_message.encode('utf-8')  # String -> Bytes
(Serileştirme)

            # UDP ile gönder
            client.sendto(json_bytes, (HOST, PORT))

            print(f"[{i+1}] Gönderildi: {json_message}")
            time.sleep(0.5)  # 500ms aralıkla gönder

    except KeyboardInterrupt:
        print("\nGönderim durduruldu.")
    finally:
        client.close()
        print("Soket kapatıldı.")

if __name__ == "__main__":
    simulate_movement()
```

3.2. Sunucu: JSON Mesajlarını Alma ve Ayrıştırma (udp_position_server.py)

```
import socket
import json

HOST = '127.0.0.1'
PORT = 12345
BUFFER_SIZE = 1024
```

```python
def process_position_message(json_data, client_addr):
    """Gelen JSON mesajını ayrıştırır ve içindeki verileri işler."""
    try:
        # JSON'ı sözlük (dict) olarak ayrıştır (Deserialization)
        message = json.loads(json_data)

        # Mesaj tipini kontrol et
        if message.get("type") == "POSITION_UPDATE":
            player_id = message["data"]["playerId"]
            pos = message["data"]["position"]
            rotation = message["data"]["rotation"]
            timestamp = message["timestamp"]

            print(f"\n[{timestamp}] Oyuncu {player_id} konumu:")
            print(f"  X: {pos['x']}, Y: {pos['y']}, Z: {pos['z']}")
            print(f"  Rotasyon: {rotation} derece")
            print(f"  Kaynak: {client_addr}")
            return True
        else:
            print(f"Bilinmeyen mesaj tipi: {message.get('type')}")
            return False

    except json.JSONDecodeError as e:
        print(f"JSON Ayrıştırma Hatası ({client_addr}): {e}")
        print(f"Alınan ham veri: {json_data}")
        return False
    except KeyError as e:
        print(f"Eksik alan hatası: {e}")
        return False

def start_udp_server():
    """UDP sunucusunu başlatır ve gelen pozisyon mesajlarını işler."""
    server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    server.bind((HOST, PORT))
    print(f"UDP Pozisyon Sunucusu {HOST}:{PORT} adresinde çalışıyor...")
    print("JSON formatında konum güncellemeleri bekleniyor.\n")

    try:
        while True:
            # Veriyi ve gönderen adresini al
            data, client_addr = server.recvfrom(BUFFER_SIZE)

            # Gelen byte'ları UTF-8 string'e çevir
            json_string = data.decode('utf-8')

            # JSON'u işle
            process_position_message(json_string, client_addr)

    except KeyboardInterrupt:
        print("\nSunucu kapatılıyor...")
    finally:
        server.close()
```

```
if __name__ == "__main__":
    start_udp_server()
```

---

4. Kodun Detaylı Analizi

4.1. Serileştirme (Serialization) Aşaması (İstemci)

1. Veri, Python sözlüğü (dict) olarak oluşturulur.
2. `json.dumps()` ile sözlük, JSON formatında bir string'e dönüştürülür.
3. `.encode('utf-8')` ile bu string, ağ üzerinden gönderilebilecek byte dizisine dönüştürülür.
4. `sendto()` ile UDP üzerinden gönderilir.

4.2. Deserialization (Ayrıştırma) Aşaması (Sunucu)

1. `recvfrom()` ile gelen byte dizisi alınır.
2. `.decode('utf-8')` ile byte'lar string'e çevrilir.
3. `json.loads()` ile JSON string'i Python sözlüğüne dönüştürülür.
4. Sözlük içindeki anahtarlara (keys) erişilerek veriler okunur ve işlenir.

4.3. JSON Mesajının Avantajları

- Kod son derece okunabilir.
- Yeni bir alan (örneğin "velocity" - hız) eklemek isterseniz, sadece mesaj yapısına ekleyin; sunucu hata vermeden yeni alanı görmezden gelir veya işleyebilir.
- Hata ayıklama (debug) çok kolaydır. `print(json_string)` ile gönderilen veriyi doğrudan görebilirsiniz.

4.4. JSON Mesajının Dezavantajları (Bu Uygulamada)

- Her paket yaklaşık 120-150 byte boyutunda. Aynı veri binary olarak gönderilseydi yaklaşık 20-25 byte olurdu.
- Saniyede 60 paket göndersek, JSON 9 KB/s, binary ise 1.5 KB/s bant genişliği harcar. Fark, büyük ölçekte çok önemlidir.

---

5. Mesaj Formatı Tasarımı Önerileri

1. **Mesaj Türünü Header'da Tutun:** Yukarıdaki örnekte `"type"` alanı body içinde. İdeal tasarımda `type` header'da tutulur, böylece body'yi parse etmeden önce mesajın türü bilinir.
2. **Uzunluk Bilgisi Ekleyin:** UDP'de gerekli değil ama TCP'de kesinlikle şarttır. Her mesajın başında 2 veya 4 byte uzunluk bilgisi gönderin.
3. **Versiyonlama (Versioning):** `"version": 1` alanı ekleyin. Gelecekte format değişirse, eski istemcilerle uyumluluğu koruyabilirsiniz.
4. **Sık Kullanılan Verilerde Binary'e Geçin:** JSON ile prototip hızlıca geliştirilir. Ürün aşamasında kritik veriler (konum, atış) binary formatına dönüştürülür.

---

6. Binary Alternatifi (Kavramsal Gösterim)

Aynı veriyi binary olarak göndermek isteseydik, bir "packer" (paketleyici) yazmamız gerekirdi:

```
import struct  # Python'da binary veri paketleme

def pack_position_binary(player_id, x, y, z, rotation):
    # Format: < = little-endian, I = unsigned int, f = float
    # I (4 byte) + f (4 byte) * 4 = 20 byte
    return struct.pack('<Iffff', player_id, x, y, z, rotation)

# Gönderim
binary_data = pack_position_binary(101, 125.5, 75.2, 0.0, 90.0)
client.sendto(binary_data, (HOST, PORT))
```

Sunucuda ise:

```
player_id, x, y, z, rotation = struct.unpack('<Iffff', data)
```

Bu yöntemle:

- Mesaj boyutu: JSON ~150 byte, Binary **20 byte**.
- İşlem hızı: Binary parse işlemi JSON'a göre ~10x daha hızlı.

İleri Çalışma

1. **JSON Mesajına Tarih ve Hız Bilgisi Ekleyin:** `"velocity"` alanı ekleyin (Vx, Vy, Vz). Sunucuyu bu yeni alanı da okuyacak şekilde güncelleyin.
2. **Binary Format Dönüşümü:** Yukarıdaki `struct.pack()` örneğini kullanarak JSON yerine binary gönderim yapan bir istemci ve alıcı sunucu yazın. İki yöntemin bant genişliği kullanımını karşılaştırın.
3. **TCP Üzerinde JSON Gönderimi:** TCP'de `send()` ve `recv()` kullanarak, mesaj başında 4 byte uzunluk bilgisi olacak şekilde JSON mesajları gönderen bir sistem tasarlayın.

JSON, prototipleme, debug ve esnek mesajlaşma için vazgeçilmezdir. Ancak gerçek zamanlı oyunlarda, özellikle saniyede yüzlerce paket gönderildiğinde, bant genişliği ve işlemci yükü kritik hale gelir. Bu noktada binary formatlar devreye girer. Bir oyun geliştiricisi, hangi verinin hangi formatta gönderileceğine, verinin önemine, gönderim sıklığına ve performans gereksinimlerine göre karar vermelidir. İdeal yaklaşım, JSON ile hızlı prototipleme yapıp, daha sonra darboğaz olan kısımları binary formatına dönüştürmektir.